



RG Ambiental - Documentação técnica do sistema

Última atualização: 2026-05-18

Repositório: [rh-ambiental-sistema](#)

URL de produção (oficial): <https://rh-ambiental-sistema.vercel.app>

Versão do pacote: **1.2.1** (exibida na UI como **R maj.min.patch**, ver secção 12)

Tipo: SPA React + Supabase (Auth, Postgres, Storage, Edge Functions) + PWA.

Subpáginas relacionadas:

- [Servidor Offline — Infraestrutura Local](#) — **guia unificado** (spec + **carrinho Pichau R\$ 5.000** + implantação)
- [Precificação em Camadas — Planos RG Ambiental](#) — planos SaaS cloud (Básico / Pro / Enterprise)

1. Resumo executivo

O **RG Ambiental Sistema** é a aplicação web interna da RG Ambiental para gestão do fluxo operacional de coleta de resíduos: cadastros (clientes, motoristas, veículos), programação, MTR, pesagem, comprovantes, conferência, tickets, aprovação da diretoria, faturamento, envio de NF por e-mail, financeiro (contas a receber/pagar), pós-venda, utilizadores e chat interno.

A interface é uma **Single Page Application** em React 19, com rotas protegidas por sessão Supabase, restrição por **cargo** e, opcionalmente, por **lista de páginas** por utilizador. O backend é o **Supabase** (PostgreSQL com RLS, Auth, Storage e Edge Functions em Deno).

2. Stack e arquitetura

Camada	Tecnologia	Notas
Frontend	React 19, TypeScript, Vite 8	Entry: <code>src/main.tsx</code> → <code>App-NEXUS.tsx</code>
Roteamento	react-router-dom 7	Lazy loading com retry (<code>lazyWithRetry</code>)
UI / gráficos	CSS custom (<code>index-NEXUS.css</code>), Recharts, Leaflet	Mapas e dashboards
PDF / Excel	jsPDF, jspdf-autotable, xlsx	Relatórios e exportações
Backend	Supabase	Project ref: <code>kmuitineaiuegpbpczjc</code>
Auth	Supabase Auth (e-mail/senha)	Perfil em <code>public.usuarios</code>
API de dados	PostgREST + RLS	Políticas por cargo (<code>20260425120000_rls_por_cargo_core_fluxo.sql</code>)
Funções serverless	Supabase Edge Functions (Deno)	Admin de utilizadores + envio de NF
PWA	vite-plugin-pwa / Workbox	Atualização com prompt (<code>registerType: prompt</code>)
Hospedagem frontend	Vercel	Deploy: <code>npm run deploy:vercel</code>
Node	>= 20.19.0	Definido em <code>package.json</code> → <code>engines</code>

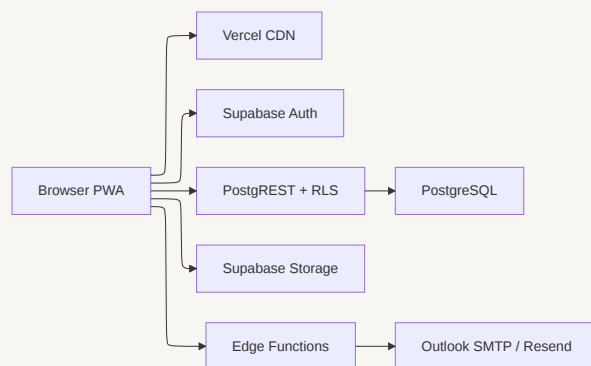
Diagrama simplificado

flowchart LR

```

Browser["Browser PWA"] --> Vercel["Vercel CDN"]
Browser --> SupabaseAuth["Supabase Auth"]
Browser --> PostgREST["PostgREST + RLS"]
Browser --> Storage["Supabase Storage"]
Browser --> EdgeFn["Edge Functions"]
EdgeFn --> SMTP["Outlook SMTP / Resend"]
PostgREST --> Postgres["PostgreSQL"]

```



3. Estrutura principal do repositório

```

rh-ambiental-sistema/
package.json          # versão semver + scripts
vite.config.ts        # PWA, VITE_APP_VERSION, cache Supabase
.env.example          # variáveis Vite e notas de deploy
public/               # assets estáticos, version.json (build)
scripts/              # seeds, deploy Vercel, migrações SQL
src/
  main.tsx            # bootstrap + ErrorBoundary
  App-NEXUS.tsx       # rotas e guards
  pages/              # ecrãs por módulo
  layouts/MainLayout.tsx # menu lateral
  lib/                # supabase, permissões, rotas
  contexts/           # perfil, chat, presença
  components/
supabase/
  migrations/         # DDL incremental (timestamped)
  functions/          # Edge Functions
  .secrets.edge.env.example
  
```

Variantes **-NEXUS** em ficheiros (`App-NEXUS.tsx` , `Login-NEXUS.tsx` , etc.) são a linha activa de produção; ficheiros sem sufixo podem existir como legado ou espelho.

4. Módulos e rotas

Rotas canónicas definidas em `src/lib/paginasSistema.ts` e protegidas em `App-NEXUS.tsx` com `NEXUS_CARGOS_POR_ROTA` (`src/lib/nexusCargosPorRota.ts`).

Rota	Módulo	Grupo no menu
/bem-vindo	Boas-vindas	Visão geral (sempre acessível)
/dashboard	Dashboard operacional ou executivo	Visão geral
/clientes	Clientes	Cadastros
/motoristas	Motoristas	Cadastros
/representantes-rg	Representantes RG	Cadastros
/caminhoes	Veículos	Cadastros
/programacao	Programação de coletas	Fluxo operacional
/mtr	MTR (Manifesto)	Fluxo operacional
/controle-massa	Pesagem e ticket	Fluxo operacional
/comprovantes-descarte	Comprovante de descarte	Fluxo operacional
/conferencia-transporte	Conferência de transportes	Fluxo operacional
/checklist-transporte	Checklist de transportes	(fora do menu principal; rota protegida)
/ticket-operacional	Ticket operacional	(fora do menu principal)
/aprovacao	Aprovação diretoria	(fora do menu principal)
/faturamento	Faturamento operacional	Faturamento
/faturamento/regras-preco	Regras de preço	Faturamento
/envio-nf	Envio de NF por e-mail	Faturamento
/financeiro	Financeiro (hub)	Financeiro
/financeiro/contas-receber	Contas a receber	Financeiro
/financeiro/contas-pagar	Contas a pagar	Financeiro
/pos-venda	Pós-venda	Pós-venda
/usuarios	Gestão de utilizadores	Sistema
/chat	Chat interno	(flutuante + rota)

Rotas com parâmetro (/mtr/:id , /controle-massa/:id , etc.) usam **prefix match** no menu.

5. Fluxo operacional (negócio)

Sequência típica de uma **coleta** (`public.coletas`), com estados sincronizados por migrações e triggers:

1. **Programação** — agenda coleta, cliente, veículo/motorista.
2. **MTR** — documentação/manifesto (`mtrs` , detalhes JSONB).
3. **Pesagem** — controle de massa / ticket de balança.
4. **Comprovante de descarte** — registo e anexos.
5. **Conferência de transporte** — validação documental.
6. **Checklist de transporte** — formulário motorista/logística (`checklist_transporte`).
7. **Ticket operacional** — após conferência (`tickets_operacionais`).
8. **Aprovação diretoria** — `aprovacoes_diretoria` (aprovado / ajuste solicitado).
9. **Faturamento** — regras de preço, `faturamento_registros` , view `vw_faturamento_resumo` .
10. **Envio NF** — e-mail via Edge Function `send-nf-email` , log em `nf_envios_log` .
11. **Financeiro** — contas a receber/pagar, parcelas, auditoria.

Permissões de mutação por ecrã: `src/lib/workflowPermissions.ts` (UI); **RLS no Supabase** é a barreira final.

6. Cargos e controlo de acesso

Cargos canónicos

Desenvolvedor, Administrador, Financeiro, Operacional, Logística, Balanceiro, Diretoria, Faturamento, Comercial, Visualizador — definidos em `CARGO_NEXUS` .

Duas camadas de restrição

1. **Por cargo** — cada rota tem lista de cargos em `NEXUS_CARGOS_POR_ROTA` . Ex.: Operacional **não** acede a Faturamento, Financeiro, Pós-venda nem Utilizadores; Logística só ao fluxo operacional do menu.

2. **Por páginas permitidas** — coluna `usuarios.paginas_permitidas` (array de prefixos). Cargo **Visualizador** exige lista explícita; sem lista só vê `/bem-vindo`.

Bypass de páginas (gestão)

E-mails que ignoram `paginas_permitidas` : `cavalcantersc07@gmail.com` , `gestores@rgambiental.com` , mais `VITE_PAGINAS_BYPASS_EMAILS` / secret `PAGINAS_BYPASS_EMAILS` na Edge Function.

Operações sensíveis

- Criar/excluir utilizadores: Administrador ou Desenvolvedor (`workflowPermissions` , Edge Functions admin).
- Editar utilizadores: também Diretoria e Financeiro (não operacional).
- Logística: **proibida** de excluir registos em qualquer módulo.
- Dashboard executivo (BI): Diretoria e perfis tipo Administrador.

7. Base de dados (Supabase / PostgreSQL)

Tabelas principais (seleção)

Tabela	Função
<code>coletas</code>	Entidade central do fluxo operacional
<code>programacoes</code> , <code>mtrs</code>	Agenda e manifestos
<code>clientes</code> , <code>motoristas</code> , <code>caminhoes</code> , <code>representantes_rg</code>	Cadastros
<code>usuarios</code>	Perfil app (cargo, foto, páginas, presença)
<code>checklist_transporte</code> , <code>conferencia_transporte</code> , <code>conferencia_operacional</code>	Etapas de conferência/checklist
<code>tickets_operacionais</code> , <code>aprovacoes_diretoria</code>	Ticket e aprovação
<code>faturamento_registros</code> , <code>faturamento_precos_regras</code>	Faturamento
<code>vw_faturamento_resumo</code>	View agregada para ecrã de faturamento

Tabela	Função
<code>contas_receber</code> , <code>contas_pagar</code> , <code>financeiro_*</code>	Módulo financeiro
<code>nf_envios_log</code>	Histórico de envios de NF
<code>comprovantes_descarte</code> , <code>residuos</code>	Descarte e catálogo
<code>chat_conversas</code> , <code>chat_mensagens</code> , <code>chat_participantes</code>	Chat interno

Migrações

Ficheiros em `supabase/migrations/` com prefixo `YYYYMMDDHHMMSS_`. Aplicar via Supabase CLI (`db push`) ou script local:

```
npm run db:apply:sql -- supabase/migrations/<ficheiro>.sql (requer DATABASE_URL ou SUPABASE_DB_PASSWORD ).
```

RLS

Política central em `20260425120000_rls_por_cargo_core_fluxo.sql` e migrações subsequentes por módulo. **Nunca** expor `SUPABASE_SERVICE_ROLE_KEY` no frontend.

8. Edge Functions

Função	Propósito
<code>admin-create-user</code>	Criar utilizador Auth + registo <code>usuarios</code>
<code>admin-update-user</code>	Atualizar perfil / credenciais
<code>admin-delete-user</code>	Remover utilizador
<code>admin-set-user-pages</code>	Definir <code>paginas_permitidas</code>
<code>send-nf-email</code>	Envio de NF (Outlook SMTP ou Resend)

CORS partilhado: `supabase/functions/_shared/cors.ts` + secret `EDGE_FUNCTION_ALLOWED_ORIGINS` .

Deploy: `npm run functions:deploy:admin-all` , `npm run functions:deploy:send-nf-email` .

Segredos: `supabase/.secrets.edge.env` → `npm run supabase:secrets:edge` .

9. Variáveis de ambiente

Frontend (Vercel / `.env`)

Variável	Obrigatória	Descrição
<code>VITE_SUPABASE_URL</code>	Sim	URL do projeto Supabase
<code>VITE_SUPABASE_ANON_KEY</code>	Sim	Chave anon/publicável
<code>VITE_APP_ORIGIN</code>	Recomendada	Origem oficial (produção)
<code>VITE_PAGINAS_BYPASS_EMAILS</code>	Não	E-mails extra com bypass de páginas
<code>VITE_FATURAMENTO_RESUMO_DESDE_DIAS</code>	Não	Limita janela da view de faturamento
<code>VITE_SUPOORTE_USER_ID</code> / <code>VITE_SUPOORTE_EMAIL</code>	Não	Destino do balão de suporte → chat

`VITE_APP_VERSION` e `VITE_APP_BUILD_STAMP` são injectados no build a partir de `package.json` e `public/version.json`.

Supabase Auth

Em **Authentication** → **URL Configuration**, incluir `https://rh-ambiental-sistema.vercel.app` e `/**` nos redirects.

Edge (segredos Supabase, não Vite)

Ver `supabase/.secrets.edge.env.example`: `OUTLOOK_*`, `RESEND_API_KEY`, `EMAIL_PROVIDER`, `NF_EMAIL_FROM`, `PAGINAS_BYPASS_EMAILS`, etc.

10. Desenvolvimento local

```
npm install
# Copiar .env.example → .env e preencher VITE_SUPABASE_*
npm run dev           # Vite na porta 4173
npm run build         # build:brand + version.json + tsc + vit
e build
npm run preview       # preview na porta 4173
npm test              # vitest
npm run lint           # eslint
```


Em `localhost`, service workers são desregistrados automaticamente (`main.tsx`) para evitar cache stale durante desenvolvimento.

11. Deploy e versão

Vercel (frontend)

1. Branch `main` ou `npm run deploy:vercel` (requer `VERCEL_TOKEN` e `npx vercel link`).
2. Alias de produção: <https://rh-ambiental-sistema.vercel.app> (`src/lib/officialSiteUrl.ts`).
3. **Regra de produto:** antes de cada release que deve mudar o número na UI, incrementar `"version"` em `package.json` (patch/minor/major). O build gera `public/version.json` → `VITE_APP_VERSION` → texto **R** `x.y.z` em `src/lib/appDisplayVersion.ts` (com incremento local opcional via balão PWA).

Supabase

- `npm run supabase:link` (uma vez)
- Migrações: CLI ou `db:apply:sql`
- Functions + secrets conforme secção 8

12. PWA e cache

- Manifest: `RG Ambiental`, tema `#0f766e`, ícones em `public/assets/logo/`.
- Service Worker: **não** faz cache de Auth/REST/Functions Supabase (`NetworkOnly` em `vite.config.ts`) para evitar fugas entre sessões.
- Storage público: `StaleWhileRevalidate`.
- Atualização: prompt ao utilizador; `skipWaiting: false` até confirmar no aviso.

13. Scripts úteis (npm)

Script	Uso
<code>seed:*</code>	Dados de demonstração / histórico (coletas, clientes, motoristas)

Script	Uso
<code>reset:coletas-operacao</code>	Limpar dados operacionais de teste
<code>db:apply:sql</code>	Aplicar ficheiro SQL à BD remota
<code>import:clientes:xlsx</code>	Importação de clientes
<code>admin:create-pateta</code> / <code>admin:create-teste</code>	Criar admin local via script
<code>deploy:vercel</code>	Deploy produção sem browser

14. Integrações externas

- **Supabase** — Auth, BD, Storage, Realtime (chat/presença onde aplicável).
- **Resend** ou **Outlook SMTP** — envio de NF (`send-nf-email`). Resend recomendado em produção (SMTP Microsoft pode dar timeout nos datacenters Supabase).
- **Google Fonts** — cache PWA.
- **Leaflet** — mapas em cadastros/operacional.
- **Vercel** — hosting e CI de deploy do frontend.

15. Segurança e boas práticas

- Usar apenas `VITE_SUPABASE_ANON_KEY` no cliente; RLS obrigatório para dados sensíveis.
- Service Role **só** em scripts/Edge Functions no servidor.
- Validar CORS das Edge Functions em produção (`EDGE_FUNCTION_ALLOWED_ORIGINS`).
- Não commitar `.env` , `supabase/.secrets.edge.env` , `.vercel/` , tokens Vercel.
- Testes: `src/lib/paginasSistema.test.ts` e outros em Vitest.

16. Riscos e pontos de atenção

1. **Duplicidade NEXUS vs legado** — confirmar que alterações vão para ficheiros `-NEXUS` usados em produção.

2. **PWA após deploy** — utilizadores podem precisar de recarregar após update (ErrorBoundary trata chunk failed).
3. **SMTP Outlook** — pode falhar na Edge; preferir Resend com domínio verificado.
4. **Visualizador sem páginas** — bloqueio intencional; configurar `paginas_permitidas` na gestão de utilizadores.
5. **Migrações** — aplicar na ordem; testar em staging antes de produção.
6. **Versão na UI** — sem bump em `package.json`, o número oficial da release não acompanha o deploy.

17. Checklist rápido (release)

- ☐ `package.json` version incrementada
- ☐ `npm run build` e `npm test` sem erros
- ☐ Migrações Supabase aplicadas (se houver)
- ☐ Edge Functions deployadas (se alteradas)
- ☐ Variáveis Vercel e secrets Supabase actualizados
- ☐ Auth redirect URLs incluem origem de produção
- ☐ Smoke test: login, uma coleta no fluxo, faturamento, envio NF (staging)
- ☐ Confirmar número **Versão do sistema** na UI após deploy

 Servidor Offline — Infraestrutura Local

 Precificação em Camadas — Planos RG Ambiental